Sistema de Rotas Internas Utilizando Grafos

Curso: Engenharia de Software
Disciplina: Árvores e Grafos
Instituição: UniEVANGÉLICA
Professor: Renato Luan

Integrantes: Jennifer Costa Silva (2411299), João Gabriel Serra (2410208), Gabriel Botelho de Siqueira (2411708) e Victor Augusto de Sousa Silva (2412043)

1. Introdução

No desenvolvimento de sistemas computacionais, é comum lidar com problemas que envolvem caminhos, conexões e relações entre diferentes elementos. Para representar esse tipo de estrutura de forma eficiente, utilizam-se grafos, que são estruturas de dados amplamente aplicadas em diversas áreas da computação.

Grafos são utilizados em sistemas de navegação, redes sociais, roteamento de dados na internet e até em jogos digitais. Sempre que existe a necessidade de representar pontos interligados e encontrar caminhos entre eles, grafos se tornam uma solução natural e poderosa.

Aplicativos de navegação, como mapas digitais e sistemas de GPS, utilizam algoritmos baseados em grafos para calcular rotas e identificar o menor caminho entre dois pontos. Esses algoritmos analisam diferentes possibilidades de percurso até encontrar a alternativa mais eficiente.

Com base nesse conceito, este projeto teve como objetivo desenvolver um sistema simples de rotas internas que simula o mapa de um Campus universitário. O sistema utiliza grafos para representar os locais e suas conexões, permitindo calcular o menor caminho entre dois pontos escolhidos pelo usuário.

Além da implementação prática, o projeto busca reforçar a compreensão dos conceitos estudados na disciplina, demonstrando como a teoria das estruturas de dados pode ser aplicada na resolução de problemas reais.

2. Conceitos Fundamentais

Para o desenvolvimento do sistema, foi necessário compreender alguns conceitos importantes relacionados à teoria dos grafos.

Um **grafo** é uma estrutura formada por dois componentes principais: vértices e arestas. Os vértices representam os pontos ou entidades do sistema, enquanto as arestas representam as conexões entre esses pontos.

No contexto deste projeto, cada local do Campus foi representado como um vértice do grafo, enquanto os caminhos que ligam esses locais foram representados como arestas.

Além disso, utilizamos **grafos ponderados**, nos quais cada aresta possui um peso associado. Esse peso representa a distância entre dois locais e é utilizado pelo algoritmo para calcular o menor caminho possível.

Outra característica importante é que o grafo utilizado no projeto é **não direcionado**, o que significa que as conexões podem ser percorridas em ambos os sentidos. Isso representa de forma mais realista a movimentação dentro de um Campus, onde normalmente é possível caminhar de um ponto ao outro e retornar pelo mesmo caminho.

Também foi necessário escolher uma forma eficiente de armazenar o grafo na memória do sistema. Existem duas formas principais de representação:

- Matriz de adjacência
- Lista de adjacência

Neste projeto foi utilizada **lista de adjacência**, pois essa estrutura consome menos memória quando o número de conexões é relativamente pequeno, além de facilitar a navegação entre os vértices conectados.

3. Algoritmo de Dijkstra

Para calcular o menor caminho entre dois pontos do grafo foi utilizado o **algoritmo de Dijkstra**, um dos algoritmos mais conhecidos para resolver problemas de caminho mínimo.

Esse algoritmo funciona explorando progressivamente os vértices do grafo, sempre priorizando aqueles que possuem a menor distância acumulada a partir do ponto inicial.

Inicialmente, o algoritmo atribui distância zero ao ponto de partida e distância infinita para todos os demais vértices. Em seguida, ele começa a analisar os vizinhos do ponto atual, atualizando as distâncias sempre que encontra um caminho mais curto.

Durante a execução, o algoritmo mantém registro das menores distâncias conhecidas e também armazena qual vértice foi utilizado para chegar até determinado ponto. Isso permite reconstruir o caminho completo ao final da execução.

O processo continua até que todos os caminhos possíveis tenham sido avaliados ou até que o destino seja alcançado.

Devido à sua eficiência e simplicidade, o algoritmo de Dijkstra é amplamente utilizado em sistemas de navegação, planejamento de rotas e redes de comunicação.

4. Planejamento do Sistema

Para tornar o projeto mais próximo de uma situação real, foi definido um pequeno conjunto de locais representando diferentes pontos de um Campus universitário.

Os locais utilizados no sistema foram:

Entrada
Biblioteca
Cantina
Bloco A
Bloco B
Secretaria
Laboratório

Cada um desses locais representa um vértice do grafo.

Em seguida, foram definidas as conexões entre os pontos, representando os caminhos existentes entre os locais. Para cada conexão foi atribuída uma distância aproximada em metros.

Essas distâncias foram utilizadas como peso das arestas do grafo, permitindo que o algoritmo calcule o caminho mais curto entre dois pontos.

5. Implementação do Sistema

A implementação do sistema foi realizada utilizando a linguagem de programação Python, escolhida por sua simplicidade, clareza e ampla utilização no ensino de algoritmos e estruturas de dados.

Foi criada uma classe chamada **Grafo**, responsável por armazenar os vértices e suas conexões. Dentro dessa classe foram implementados métodos responsáveis por:

- adicionar novos locais ao grafo
- conectar dois locais com uma determinada distância
- calcular o menor caminho entre dois pontos utilizando o algoritmo de Dijkstra

A estrutura utilizada para armazenar as conexões foi uma lista de adjacência, representada por um dicionário em Python. Nesse dicionário, cada vértice armazena uma lista contendo os vizinhos conectados e as respectivas distâncias.

O algoritmo de Dijkstra foi implementado utilizando uma **fila de prioridade**, que permite sempre selecionar o vértice com menor distância acumulada durante a execução.

O sistema também possui uma interação simples com o usuário através do terminal. O usuário informa o ponto de partida e o destino desejado, e o programa calcula automaticamente o menor caminho entre esses pontos.

codigo_arvoresgrafos.py

C: > Users > jenni > OneDrive > Documentos > faculdade > codigo_arvoresgrafos.py > ...

```python
import heapq


class Grafo:
    def __init__(self):
        # Dicionário que armazena os vértices e suas conexões
        # Exemplo: {"Entrada": [("Biblioteca", 50), ("Cantina", 30)]}
        self.vertices = {}

    def adicionar_local(self, nome):
        """Adiciona um novo local ao grafo"""
        if nome not in self.vertices:
            self.vertices[nome] = []

    def conectar_locais(self, origem, destino, distancia):
        """
        Cria uma conexão entre dois locais.
        Como o grafo é não direcionado,
        a conexão vale nos dois sentidos.
        """
        self.vertices[origem].append((destino, distancia))
        self.vertices[destino].append((origem, distancia))

    def menor_caminho(self, inicio, destino):
        """
        Implementação do algoritmo de Dijkstra
        para encontrar o menor caminho entre
        dois pontos do grafo.
        """

        # Inicializa todas as distâncias como infinito
        distancias = {local: float("inf") for local in self.vertices}
        distancias[inicio] = 0

        # Guarda o caminho percorrido
        anteriores = {local: None for local in self.vertices}
```

```python

        # Fila de prioridade
        fila = [(0, inicio)]

        while fila:
            distancia_atual, local_atual = heapq.heappop(fila)

            # Se chegou ao destino, pode parar
            if local_atual == destino:
                break

            for vizinho, peso in self.vertices[local_atual]:
                nova_distancia = distancia_atual + peso

                # Atualiza apenas se encontrou um caminho menor
                if nova_distancia < distancias[vizinho]:
                    distancias[vizinho] = nova_distancia
                    anteriores[vizinho] = local_atual
                    heapq.heappush(fila, (nova_distancia, vizinho))

        # Reconstruindo o caminho final
        caminho = []
        atual = destino

        while atual is not None:
            caminho.insert(0, atual)
            atual = anteriores[atual]

        # Se não houver caminho possível
        if distancias[destino] == float("inf"):
            return None, None

        return caminho, distancias[destino]


```

```python

# ========================
# CRIAÇÃO DO SISTEMA
# ========================

grafo = Grafo()

# Lista de locais do campus
locais = [
    "Entrada",
    "Biblioteca",
    "Cantina",
    "Bloco A",
    "Bloco B",
    "Secretaria",
    "Laboratório"
]

# Adiciona os locais ao grafo
for local in locais:
    grafo.adicionar_local(local)

# Conexões entre os locais (origem, destino, distância em metros)
conexoes = [
    ("Entrada", "Biblioteca", 50),
    ("Entrada", "Cantina", 30),
    ("Cantina", "Bloco A", 20),
    ("Bloco A", "Bloco B", 25),
    ("Bloco B", "Laboratório", 40),
    ("Biblioteca", "Secretaria", 35),
    ("Secretaria", "Laboratório", 30)
]

# Conecta os locais
for origem, destino, distancia in conexoes:
    grafo.conectar_locais(origem, destino, distancia)

```

```python

# ========================
# INTERAÇÃO COM O USUÁRIO
# ========================

print("=== Sistema de Rotas Internas ===")
print("\nLocais disponíveis:")

for local in locais:
    print("-", local)

inicio = input("\nDigite o ponto de partida: ")
destino = input("Digite o destino: ")

if inicio not in grafo.vertices or destino not in grafo.vertices:
    print("\nLocal inválido. Verifique os nomes digitados.")
else:
    caminho, distancia_total = grafo.menor_caminho(inicio, destino)

    if caminho is None:
        print("\nNão existe caminho entre os pontos informados.")
    else:
        print("\nMenor caminho encontrado:")
        print(" -> ".join(caminho))
        print("Distância total:", distancia_total, "metros")
```

6. Simulação do Sistema

Para demonstrar o funcionamento do sistema, foram realizados testes utilizando diferentes combinações de origem e destino.

Um exemplo de execução ocorre quando o usuário escolhe **Entrada** como ponto inicial e **Laboratório** como destino.

O sistema analisa todas as conexões disponíveis no grafo e calcula o menor caminho possível.

O resultado obtido foi:

Entrada → Cantina → Bloco A → Bloco B → Laboratório

A distância total percorrida nesse trajeto foi de **115 metros**.

Esse resultado demonstra que o algoritmo foi capaz de analisar diferentes caminhos e selecionar aquele com menor distância total.

Durante os testes realizados, o sistema apresentou resultados consistentes, identificando corretamente o menor percurso entre os pontos escolhidos.

```
botel@DESKTOP-NP48EPM MINGW64 ~/OneDrive/Desktop/.py (master)
$ C:/Users/botel/AppData/Local/Programs/Python/Python312/python.exe c:/Users/botel/OneDrive/Desktop/.py/app.py
=== Sistema de Rotas Internas ===

Locais disponíveis:
- Entrada
- Biblioteca
- Cantina
- Bloco A
- Bloco B
- Secretaria
- Laboratório

Digite o ponto de partida: Entrada
Digite o destino: Laboratório

Menor caminho encontrado:
Entrada -> Cantina -> Bloco A -> Bloco B -> Laboratório
Distância total: 115 metros

botel@DESKTOP-NP48EPM MINGW64 ~/OneDrive/Desktop/.py (master)
$
```

7. Resultados Obtidos

O sistema desenvolvido apresentou funcionamento correto em todos os testes realizados.

O algoritmo de Dijkstra foi capaz de identificar os caminhos mais curtos entre os diferentes pontos do grafo, respeitando as distâncias definidas entre os locais.

Além disso, a utilização de lista de adjacência mostrou-se eficiente para a representação do grafo, mantendo o código simples e organizado.

Outro aspecto importante observado foi que a implementação permite expandir facilmente o sistema. Novos locais e conexões poderiam ser adicionados ao grafo sem necessidade de alterações significativas na lógica principal do programa.

Isso demonstra a flexibilidade das estruturas de grafos e sua aplicabilidade em sistemas mais complexos.

8. Conclusão

A realização deste projeto permitiu aplicar na prática conceitos importantes relacionados a grafos e algoritmos de caminho mínimo.

A implementação do sistema de rotas internas demonstrou como estruturas de dados estudadas em sala de aula podem ser utilizadas para resolver problemas reais de navegação e planejamento de caminhos.

O algoritmo de Dijkstra mostrou-se uma solução eficiente para o cálculo do menor percurso entre dois pontos do grafo, mesmo quando existem múltiplas possibilidades de caminho.

Além de reforçar os conhecimentos teóricos da disciplina, o projeto também contribuiu para o desenvolvimento de habilidades importantes na área de Engenharia de Software, como organização de código, modelagem de problemas e documentação técnica.

9. Divisão de Responsabilidades

Cada integrante do grupo participou do desenvolvimento do projeto, contribuindo com diferentes etapas do trabalho:

Victor Augusto — Implementação do código e modelagem do grafo.
Gabriel Botelho — Desenvolvimento da lógica do algoritmo e testes.
Jennifer Costa — Documentação e elaboração do relatório.
João Serra — Revisão do código e preparação da apresentação.